

OWUS

 [maoyab/OWUS](#)

 english

 gemini-2.5-flash

 7bb8fe8

 Dec 9, 2025

Chapters

Overview of OWUS

Chapter 1: Ecohydrological Data Preparation

Chapter 2: Stochastic Soil Water Balance Model (SSWM)

Chapter 3: Water Use Strategy Framework

Chapter 4: Model Parameter Inference (MCMC)

Chapter 5: Scientific Visualization

Show Raw Markdown

Chapter 4: Model Parameter Inference (MCMC)

Welcome back! In [Chapter 1: Ecohydrological Data Preparation](#), we got our environmental data ready. In [Chapter 2: Stochastic Soil Water Balance Model \(SSWM\)](#), we used that data with our SSWM to simulate a plant's water budget. Then, in [Chapter 3: Water Use Strategy Framework](#), we learned about comparing different plant "game plans" by feeding varying parameters into the SSWM.

But here's a crucial question: where do those "game plan" parameters come from? How do we know what values to use for things like P_{g50} (stomatal sensitivity) or k_{x1_max} (maximum stomatal conductivity) that truly represent a specific plant at a specific site?



This chapter introduces the powerful methodology for estimating these unknown model parameters using observed data: **Model Parameter Inference (MCMC)**. Think of this process like a skilled detective using scattered clues (observed data) to piece together the most likely scenario (model parameters) that accurately explains a complex event.

What Problem Does Model Parameter Inference (MCMC) Solve? Finding the Best-Fit "Game Plan"!

Imagine you have a detailed instruction manual for building a LEGO car (our SSWM), but some crucial measurements (the parameters) are missing. You also have a picture of the finished car (your observed soil moisture data for 'AT-Neu'). Your goal is to figure out the missing measurements so that if you built the car with those measurements, it would look as much like the picture as possible.

The central problem: We want to find the specific values for unknown plant and soil parameters that make our Stochastic Soil Water Balance Model (SSWM) outputs (like the predicted soil moisture distribution) match the actual observed soil moisture data from a site as closely as possible.

A concrete example: For our 'AT-Neu' grassland site, we have observations of how wet or dry the soil typically is throughout the growing season. We want to find the values for plant parameters like:

- $Pg50$: How sensitive the plant's stomata are to drying soil.
- $Px50$: How vulnerable the plant's water transport system (xylem) is to cavitation (damage from air bubbles).
- k_{x1_max} : The maximum capacity of the plant's xylem to transport water.
- RAI : The Root Area Index, related to how much root surface is available for water uptake.

These parameters define the plant's water use "game plan." MCMC helps us systematically search for the combination of these parameters that best explains our observed soil moisture data. It's like finding the exact dimensions that make your LEGO car perfectly match the picture!

Key Concepts: The Detective's Toolkit

To understand how MCMC works, let's break down the essential concepts:

1. Parameters vs. Observations

- **Parameters:** These are the "settings" or "traits" of our model (e.g., $Pg50$, k_{x1_max}). We don't know their exact values, and we want to estimate them.

- **Observations:** These are the real-world measurements we collected (e.g., daily soil moisture values at 'AT-Neu'). These are our "clues."

2. Model Performance (Likelihood)

When we pick a set of parameters and run our SSWM, it predicts a soil moisture distribution. We need a way to tell *how good* that prediction is compared to our actual observed soil moisture. This "goodness of fit" is measured by something called **log-likelihood**.

- **High Log-Likelihood:** Means the model's predictions with these parameters closely match the observations. This is a "good" parameter set.
- **Low Log-Likelihood:** Means the predictions don't match well. This is a "bad" parameter set.

3. Parameter Space: The Map of Possibilities

Imagine a vast, multi-dimensional landscape. Each point on this landscape represents a unique combination of parameter values. Our goal is to find the "peaks" on this landscape – the parameter combinations that give the highest log-likelihood (best fit). This entire landscape of possible parameter values is called the **parameter space**.

4. Quantifying Uncertainty: More Than Just a Single Best Value

Even with the best detective work, there might be a few scenarios that explain the clues pretty well. We don't just want *one* "best-fit" value for each parameter. We want to know the *range* of values that are plausible. MCMC helps us understand this **parameter uncertainty** by giving us a whole distribution of likely parameter values, not just a single point.

5. Markov Chain Monte Carlo (MCMC): A Smart Random Walk

MCMC, specifically the Metropolis-Hastings algorithm used in OWUS, is a clever way to explore this vast parameter space. It's like a blindfolded hiker trying to find the highest point on a complex, hilly terrain.

- **It takes "steps":** The hiker takes a small, random step in any direction (proposes a new set of parameter values).
- **It evaluates its position:** After each step, it checks if the new location is "higher" (has a better log-likelihood) than the previous one.
- **It decides to "move" or "stay":**
 - If the new location is significantly higher, it almost always moves there.
 - If the new location is slightly higher, it probably moves there.
 - If the new location is lower, it *sometimes* still moves there, but with a certain probability. This is crucial! It prevents the hiker from getting stuck on a small "hilltop" and helps it explore the whole landscape to find the *true highest peak*.

- **It forms a "chain":** By repeating this process many, many times, the hiker leaves a trail (a "Markov Chain") of parameter values. Over time, this trail will spend more time in the areas with high log-likelihood, giving us a good estimate of the most likely parameter values and their uncertainty.

How to Use MCMC: Finding the Best-Fit Parameters

In `OWUS`, the MCMC process is managed by the `Processor` class, specifically through a function called `go_mhmc` in the `get_WUS_sites.py` file. This function takes your prepared site data, a list of parameters you want to estimate, and runs the MCMC algorithm.

Let's see how you would conceptually trigger this process for our 'AT-Neu' site:

```
1 import pandas as pd
2 import pickle
3 from get_WUS_sites import go_mhmc # Our MCMC function
4
5 site_id = 'AT-Neu'
6
7 # 1. Load the prepared parameters and observed soil moisture for 'AT-Neu' from Ch
8 data_path = 'DATA/WUS/WUS_input_data'
9 params_filepath = f'{data_path}/{site_id}_params.pickle' # Note: in code it's _pa
10 with open(params_filepath, 'rb') as f:
11     site_data = pickle.load(f)
12
13 # The observed soil moisture (s_obs) is stored within the loaded data dictionary
14 observed_soil_moisture = site_data['s_obs']
15
16 # 2. Define which parameters we want to infer (our "unknowns")
17 unknown_params_to_infer = ['Pg50', 'Px50', 'k_xl_max', 'RAI']
18
19 # 3. Define a list of all parameters that should be saved in the results
20 # This includes both inferred parameters and other useful metrics from SSWM
21 save_result_params = ['Px50', 'Pg50', 'k_xl_max', 'RAI', 'epsilon', 'mean_tf', 'ks
22
23 # 4. Prepare initial parameter ranges (important for the MCMC algorithm)
24 # These define the boundaries within which the parameters can vary.
25 # In OWUS, these are pre-defined in a dictionary called `p_ranges`.
26 # For simplicity, let's represent a small subset here:
27 p_ranges = {
28     'k_xl_max': [0.0001, 0.002], # Example range for k_xl_max
```

```

29     'Px50': [-10, -0.1],          # Example range for Px50
30     'Pg50': [-10, -0.1],          # Example range for Pg50
31     'RAI': [1, 80],               # Example range for RAI
32     # ... more ranges for other parameters ...
33 }
34
35 # 5. Run the MCMC inference!
36 print(f"Starting MCMC inference for site: {site_id}")
37 inferred_results = go_mhmcnc(site_data,
38                               unknown_params_to_infer,
39                               save_result_params,
40                               epsi=None, # None means we are doing 'best-fit' inference
41                               num_pl=3, # Number of independent MCMC chains to run
42                               nbins=100) # Number of bins for soil moisture probability
43
44 print(f"MCMC inference complete for site: {site_id}")
45
46 # 6. Access and interpret the results
47 # 'inferred_results' will be a dictionary containing the medians, standard deviations
48 # and full distributions (chains) for the inferred parameters.
49 # It also includes metrics like log-likelihood.
50 print(f"\nInferred median Pg50: {inferred_results['Pg50_median']:.2f} MPa")
51 print(f"Inferred median k_xl_max: {inferred_results['k_xl_max_median']*1000:.4f} cm/d")
52 print(f"Average log-likelihood: {inferred_results['loglikelihood']:.2f}")
53
54 # You can also get the full list of estimated values for a parameter
55 # These are the "chains" of accepted parameter values
56 # print(f"Sampled Pg50 values (first chain): {inferred_results['Pg50_estimates'][:100]}")
57

```

What happens here?

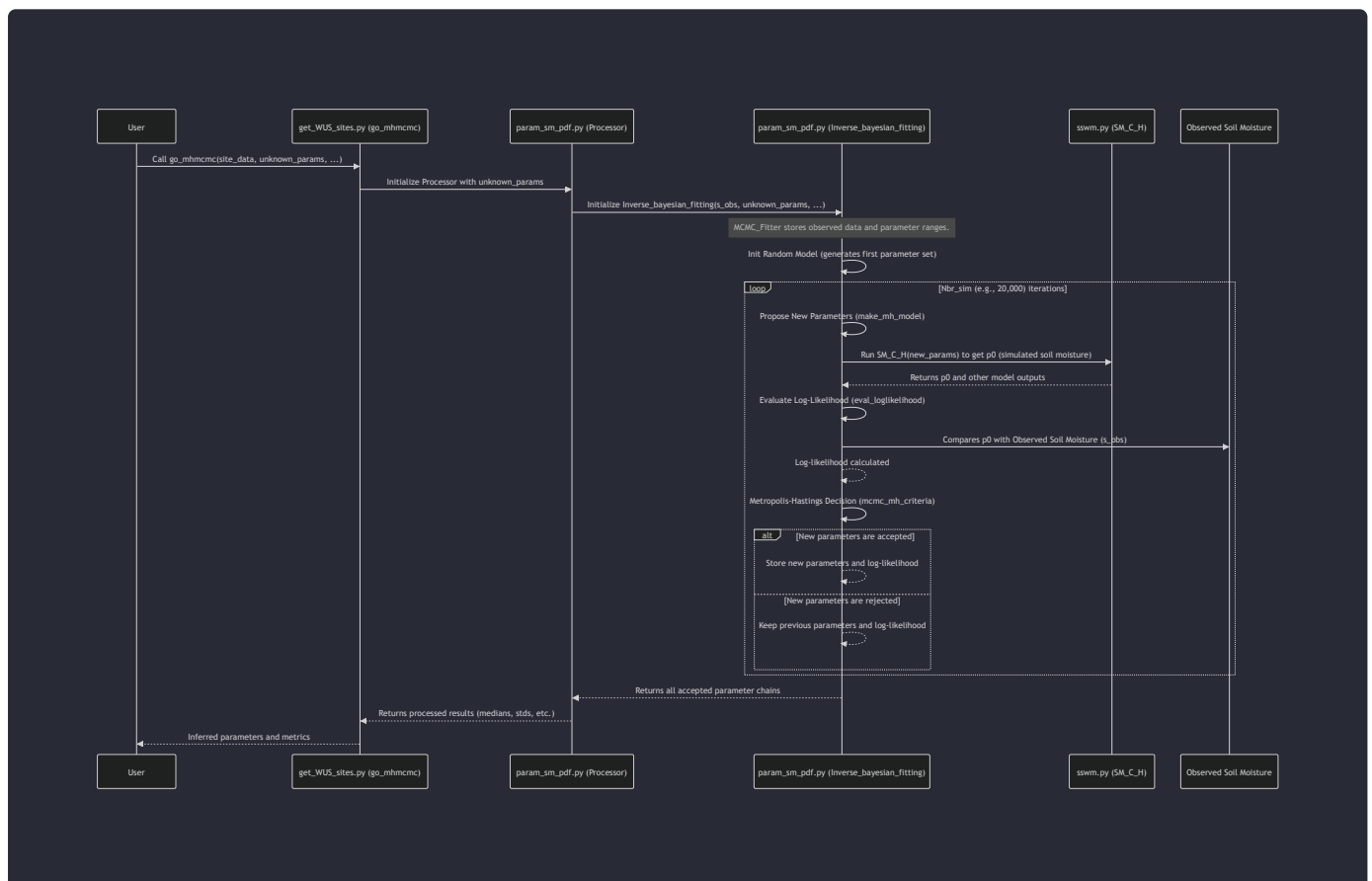
- We load the existing site data, which includes the `s_obs` (observed soil moisture values).
- We tell the `go_mhmcnc` function *which* parameters we want to find (`unknown_params_to_infer`).
- The `epsi=None` tells the function to optimize for the `log-likelihood` (best fit to observations). If `epsi` were set to `1` (as it is for "optimality" in the full `OWUS` workflow), it would try to maximize the SSWM's `epsilon` (water use efficiency) instead. This directly connects to the "data-driven" vs. "optimality-based" strategies from [Chapter 3: Water Use Strategy Framework](#).

- The `go_mhmc` function then uses MCMC to search for the best values for `Pg50` , `Px50` , `k_xl_max` , and `RAI` that make the SSWM's predicted soil moisture distribution (`p0`) match the `observed_soil_moisture` .
- The output `inferred_results` contains the statistical summaries (like median, mean, standard deviation) of the estimated parameters, along with diagnostic metrics. Importantly, it stores the entire *distribution* of accepted parameter values for each unknown.

Under the Hood: The MCMC Engine

The core MCMC logic resides in the `param_sm_pdf.py` file, within the `Inverse_bayesian_fitting` class, which is wrapped by the `Processor` class in the same file. The `go_mhmc` function from `get_WUS_sites.py` orchestrates the whole process.

Here's a simplified flow of what happens when you call `go_mhmc` :



Diving Deeper into the Code (`param_sm_pdf.py`)

Let's look at some key parts of the `Inverse_bayesian_fitting` class that implement the MCMC algorithm (simplified for clarity).

1. **The MCMC Loop (`mcmc_mh`):** This is where the "random walk" happens. It iterates many times, proposing new parameter sets and deciding whether to accept them.

```

1 # From param_sm_pdf.py (simplified)
2 class Inverse_bayesian_fitting(object):
3     # ... (initialization code) ...
4
5     def mcmc_mh(self, theta0_dict):
6         accepted = 0.
7         # Start with an initial random parameter set (theta_i) and its log-likelihood
8         (li, theta_i) = self.init_random_model(theta0_dict)
9
10        result_chain = []
11        for it in range(self.nbr_sim): # nbr_sim is the total number of steps
12            # Propose a new set of parameters and evaluate them
13            (acc, l_acc, theta_acc) = self.mcmc_mh_criteria(li, theta_i)
14
15            # Store the accepted parameters (after a 'burn-in' period to stabilize)
16            if it >= self.nbr_sim * self.burnin:
17                theta_list = [getattr(theta_acc, vv) for vv in self.save_params]
18                result_chain.append([l_acc, theta_list])
19                accepted = accepted + acc # Count how many were accepted
20
21            # Update current parameters to the accepted ones for the next step
22            li, theta_i = l_acc, theta_acc
23        return result_chain, accepted / (self.nbr_sim * (1 - self.burnin)) * 100.
24

```

This `mcmc_mh` function orchestrates the entire MCMC simulation. It repeatedly calls `mcmc_mh_criteria` to propose new steps and decide whether to move. The `result_chain` collects all the accepted parameter sets, forming the distribution we are interested in.

2. Proposing New Parameters (`make_mh_model`): When MCMC takes a "step," it needs to suggest new parameter values. This is done by adding a small random amount (noise) to the current parameter values.

```

1 # From param_sm_pdf.py (simplified)
2 def make_mh_model(self, theta0, w=0.02):
3     params_new = []
4     for param_name in self.full_param_name_list:
5         if param_name in self.unknown_params:
6             # Add random noise to unknown parameters (e.g., Pg50, Px50)
7             current_value = getattr(theta0, param_name)

```

```

8         # w is a scaling factor for the random step size
9         step_size = w * (self.p_ranges[param_name][1] - self.p_ranges[param_n
10        proposed_value = np.random.normal(current_value, step_size)
11        params_new.append(proposed_value)
12    else:
13        # Keep known parameters fixed
14        params_new.append(getattr(theta0, param_name))
15    return params_new
16

```

This function generates a `params_new` list, which is a slightly varied version of the `theta0` (current best) parameters. The variation is random but controlled by `w` (step size) and the defined `p_ranges`.

3. Evaluating Model Performance (`eval_loglikelihood`): After proposing new parameters, we need to know how well they fit the observed data. This function does that by running the SSWM and comparing its output (`p0`) to the observations (`s_obs`).

```

1  # From param_sm_pdf.py (simplified)
2  def eval_loglikelihood(self, p0_model_output, s_observed, current_params):
3      # p0_model_output is the simulated soil moisture probability distribution from
4      # s_observed is the list of actual observed soil moisture values
5
6      # This function calculates the probability of observing each s_observed value
7      # given the model's predicted distribution (p0_model_output).
8      # It involves mapping observed values to their corresponding probabilities in p
9
10     # ... (complex interpolation and lookup to find log-probability for each s_obs
11
12     log_probabilities = self.eval_logps(p0_model_output, s_observed, current_params
13
14     # The total log-likelihood is the sum of these log-probabilities
15     # A higher sum means a better fit.
16     return np.sum(log_probabilities)
17

```

This `eval_loglikelihood` function is critical. It acts as the "scorecard." It takes the SSWM's predicted soil moisture distribution (`p0_model_output`) and our actual `s_observed` data. It then

calculates a single number (the log-likelihood) that tells us how likely it is to have observed our `s_observed` data if the `p0_model_output` truly represented the system.

4. Accept/Reject Decision (`mcmc_mh_criteria`): This is the "smart" part of the random walk. It decides if the proposed new parameters are better and whether to accept them.

```
1 # From param_sm_pdf.py (simplified)
2 def mcmc_mh_criteria(self, li_current, theta_current):
3     # 1. Evaluate the proposed new parameters (l ii_new)
4     lii_new, theta_new = self.eval_mh_model(theta_current)
5
6     if lii_new > li_current:
7         # 2. If the new log-likelihood is better, always accept!
8         return [1, lii_new, theta_new] # [accepted_flag, new_likelihood, new_param]
9     elif np.random.uniform(0.0, 1.0) < np.exp(lii_new - li_current):
10        # 3. If the new log-likelihood is worse, accept with a certain probability
11        # (exp(lii_new - li_current) will be a number between 0 and 1)
12        return [1, lii_new, theta_new]
13    else:
14        # 4. Otherwise, reject and stay at the current parameters
15        return [0, li_current, theta_current]
16
```

This function implements the core Metropolis-Hastings rule. It's what allows the MCMC chain to explore the parameter space efficiently, finding areas of high likelihood while also occasionally stepping into lower likelihood areas to avoid getting stuck.

Conclusion

In this chapter, we've uncovered the detective work behind finding the hidden parameters of our Stochastic Soil Water Balance Model (SSWM). We learned that **Model Parameter Inference (MCMC)**, particularly the Metropolis-Hastings algorithm, is a powerful tool for estimating these unknown "game plan" parameters using observed ecohydrological data. By taking smart "random walks" through the parameter space, MCMC not only finds the best-fit parameter values but also quantifies their uncertainty, giving us a more complete and realistic understanding of plant water use strategies.

Now that we can prepare data, simulate water balance, define water use strategies, and infer their parameters, the final step is to make sense of all these complex results through compelling

visual stories.

Are you ready to transform numbers and parameters into meaningful scientific insights? Then let's move on to the next chapter!

[Next Chapter: Scientific Visualization](#)

Generated by [AI Codebase Knowledge Builder](#). **References:** [1], [2]

© 2025 Codebase to Tutorial. All rights reserved.

[Terms of Service](#) [Privacy Policy](#)